# A

注意到假如三个长为奇数的序列中位数相同，那这个中位数也是把三个序列拼起来的中位数。

那么合法答案一定是两个序列，只需要加数求中位数，可以用对顶堆这样的算法去做。

# B

这题我们肯定需要先求出，固定 $a$ 序列后的容量。

容易发现这只和每个数右侧比他大的元素个数有关，把这些值取 max 会得到容量。

在二分答案之后，又容易发现填数肯定是从小往大填，而且一个数是没必要填的比前缀最大值还大的，所以可以得到这些数也不是 max。

那只需要考虑除了这些填的数是否合法，这相当于对后缀有一个要求，列出来求 max 即可。

用树状数组预处理每个数后面比它大的数字个数可以做到 1log。

# C

由于我们可以在同一条边上来回移动，那么只要到每个点奇数长度和偶数长度最短路相同即可。

设这两条最短路为 $(x,y)$，其中 $x<y$，那么这样的一个点只能和 $(x-1,y-1)$ 连接，或者同时连接 $(x+1,y-1),(x-1,y+1)$。

特别的，如果 $y=x+1$，那么连接另一个 $(x,y)$ 以及一个 $(x-1,y+1)$。

我们可以把所有点按 $x+y$ 分组，同一组内的问题相对独立，按 $x$ 从小到大扫描：

首先如果存在 $(x-1,y-1)$ 那么优先连接肯定更优，否则连接 $(x-1,y+1)$ 和 $(x+1,y-1)$。

如果 $(x+1,y-1)$ 已经和当前点连接，那么继续连接 $(x-1,y+1)$ 而非 $(x-1,y-1)$，因为此时不断连接到 $(x,x+1)$，只需要一条边就能解决两个点。

模拟上述过程即可，注意特判二分图和 $1$ 有自环的情况。

时间复杂度 $O(n\log n)$。

# D

考虑如何确定一个保留方案合法。假设保留的下标序列为 $p_1,p_2,...,p_k$，则对于任意 $i\in[1,k)$，位于 $[p_i,p_{i+1}-1]$ 内的所有字符都需要能够删除得只剩 $p_i$。于是直接考虑对于一个区间 $[l,r]$，考察它能否被删除到只剩 $l$。我们可以倒着扫描整个区间，使用一个栈维护目前还未被删除的字符。对于当前进入的一个字符 $c$，不断弹出栈顶的字符直到栈顶字符不能被 $c$ 删除，然后把 $c$ 压入栈。若最后栈中只有 $l$ 处的一个字符，则该区间合法。

考虑第一个段，不难发现第一个位置的字符是一定会被保留的。枚举最靠后的一个被这个字删除的位置 $x$，则存在一种合法方案是先处理完 $x+1\sim n$ 的所有字符，然后再利用 $1$ 删除 $2\sim x$ 的所有字符。不难发现这实际上是将一个 $1\sim n$ 的问题递归到了一个 $x\sim n$ 的问题。不妨设 $f_i$ 表示 $i\sim n$ 的答案，也即从 $i$ 开始执行操作得到的最小字典序，提前预处理每个区间是否能被删除得只剩第一个，枚举即可做到 $O(n^2)$。

考虑去除无效的转移点。对于一个加入栈的元素序列，若取出其后缀进行操作，不难发现这部分元素的压入弹出操作与原先一致。因此可以直接对 $[1,n]$ 执行上述流程，设 $S_i$ 表示被 $i$ 删除的所有位置集合，则所有 $S_i$ 中的转移点都是合法的，同时根据上述结论，我们发现任取后缀的情况下，其他点也不会被 $i$ 删除，故不存在 $S_i$ 以外的有效转移点。不难发现 $S$ 大小的总和是 $O(n)$ 的，于是转移复杂度正确。

具体地，我们设 $f_i$ 表示以 $i$ 作为首个保留的位置时，最小字典序的下一个保留位置；$g_i$ 表示以 $i$ 作为首个保留位置的最小字典序，求 $f_i$ 时，找到所有 $j\in S_i$，并比较 $g_{f_j}$ 的字典序，选取最小的那一个（这是因为 $j$ 实际上是会被删除的，$i$ 的下一个字符实际上在位置 $f_j$）接在 $i$ 的字符之后作为 $g_i$ 即可。注意到 $g$ 存在拼合关系，故我们可以使用字符串哈希，并倍增找到两个串首个不同位置的方式，以此在 $O(\log n)$ 的复杂度内比较两个串的字典序，只需将 $g$ 改为倍增数组存储哈希值即可。

最终总复杂度为 $O(n\log n)$，瓶颈在每个转移处 $O(\log n)$ 比较字典序。代码实现中使用了双哈希。